

Zwielokrotnianie i spójność



Zwielokrotnianie

Zwielokrotnianie polega na utrzymywaniu wielu kopii danych (obiektów) na niezależnych serwerach

Cele zwielokrotniania

1. zwiększenie efektywności
2. zwiększenie niezawodności (i dostępności)

Główne problem – spójność

- modele spójności
- protokoły spójności
- mobilność



Po co zwielokrotniać?

Niezawodność

- odporność na awarie

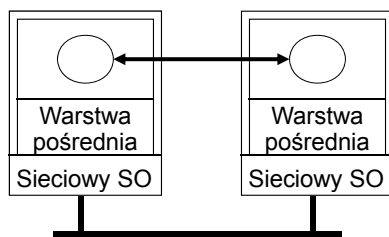
Efektywność

- współbieżny dostęp do wielu serwerów
 - równoważenie obciążenia
 - skalowalność w sensie liczbowym
- wykorzystanie bliższych serwerów
 - skalowalność w sensie geograficznym
 - mniejsze opóźnienia

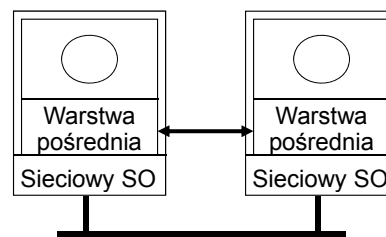


Zwielokrotnianie obiektów

a)



b)





Zwielokrotnianie a skalowalność

Kompromis

- skracanie czasu dostępu do serwerów
- utrzymywanie kopii w stanie spójnym
 - koszt utrzymywania nieużywanych kopii
- spójność odwzorowująca system scentralizowany
 - transakcyjna aktualizacja wszystkich kopii
 - globalna synchronizacja
- osłabienie modelu spójności
 - charakter aplikacji i zwielokrotnianych danych

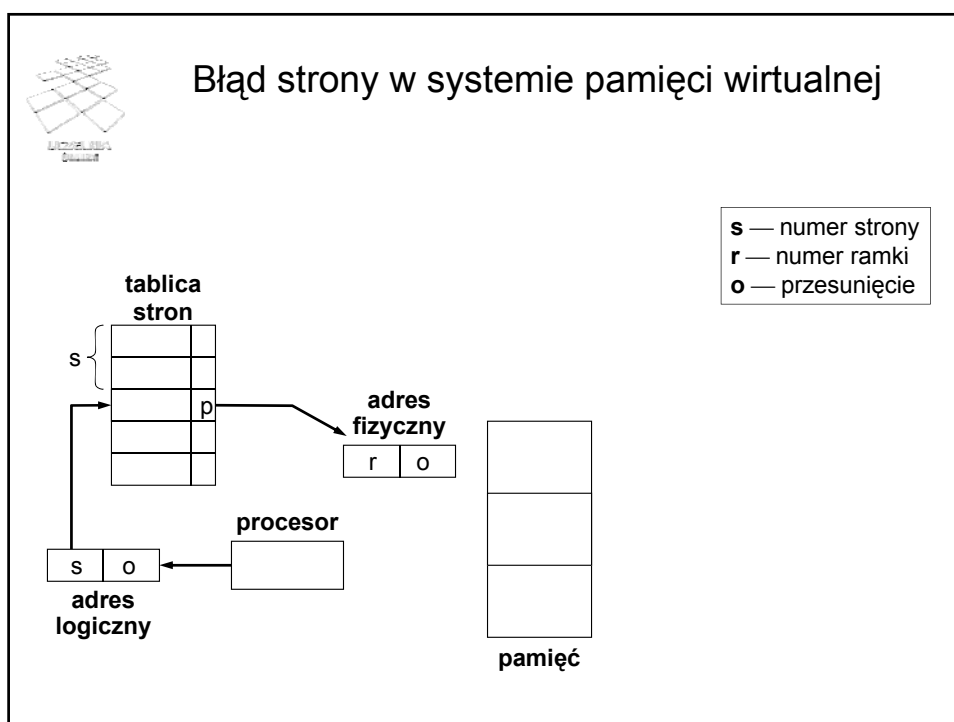
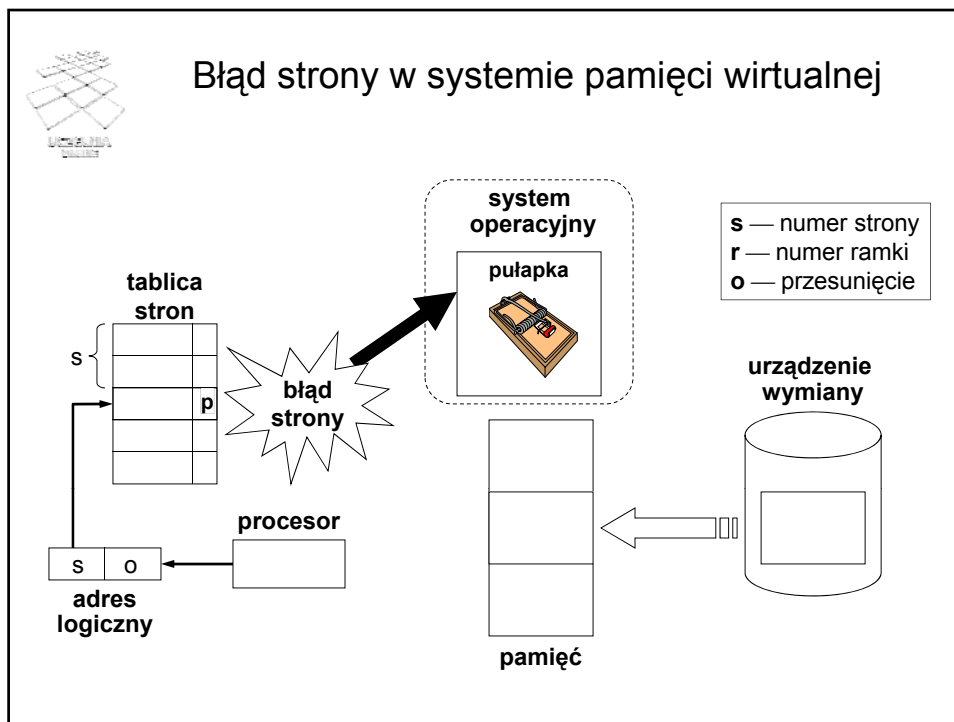


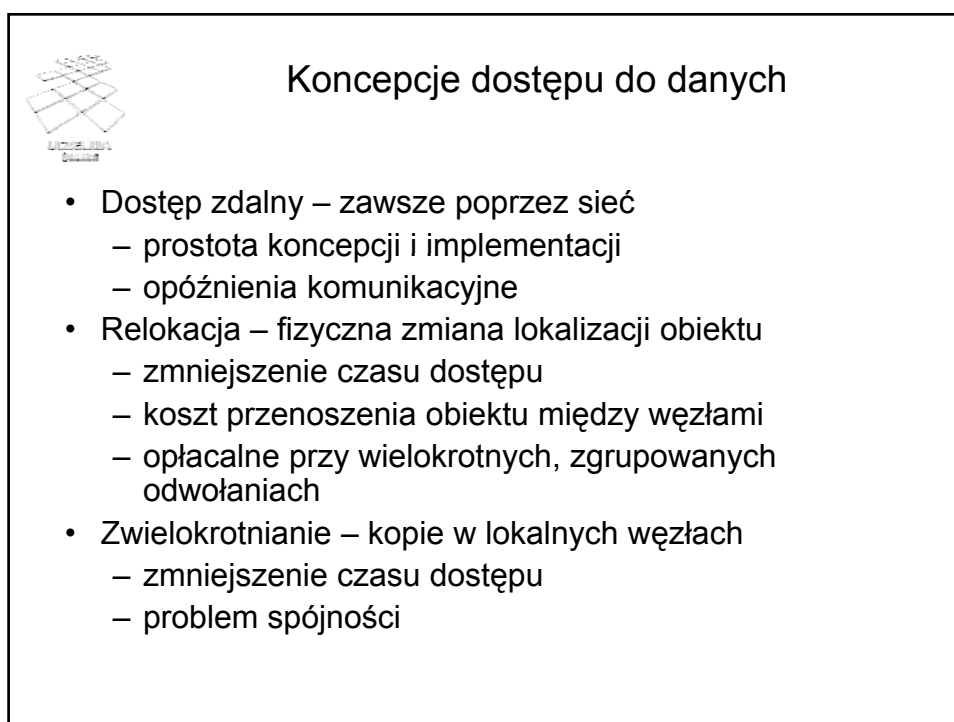
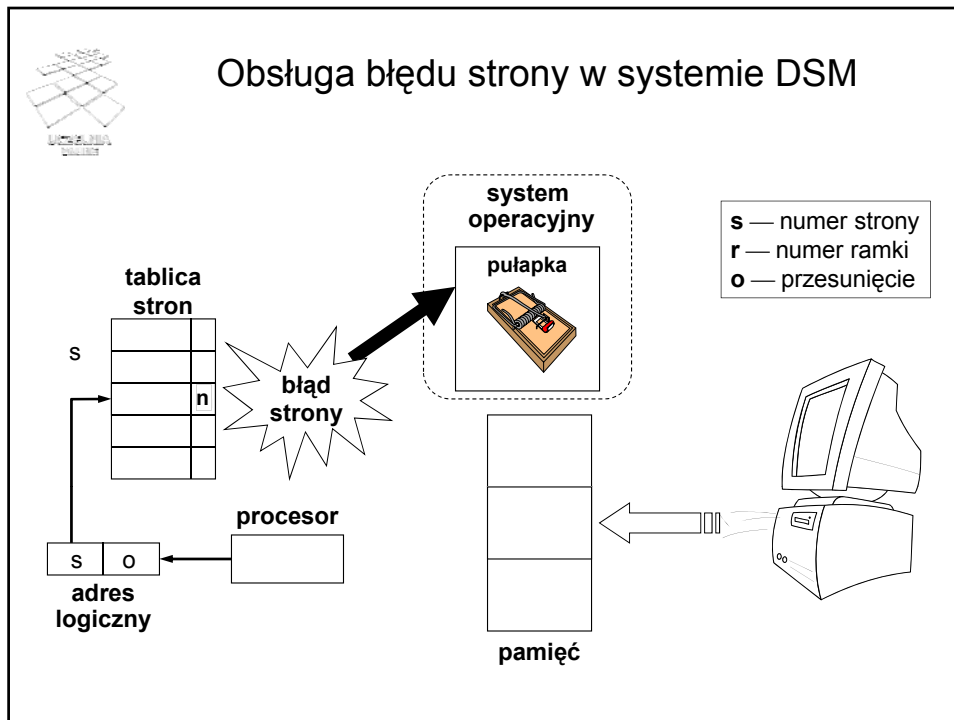
Rozproszona pamięć dzielona

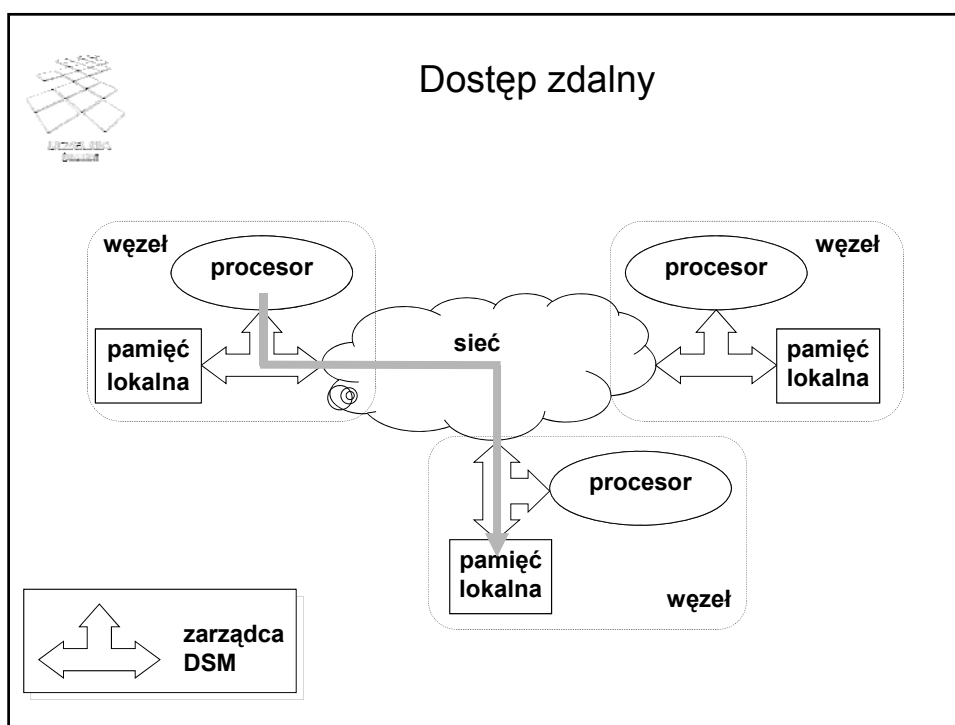
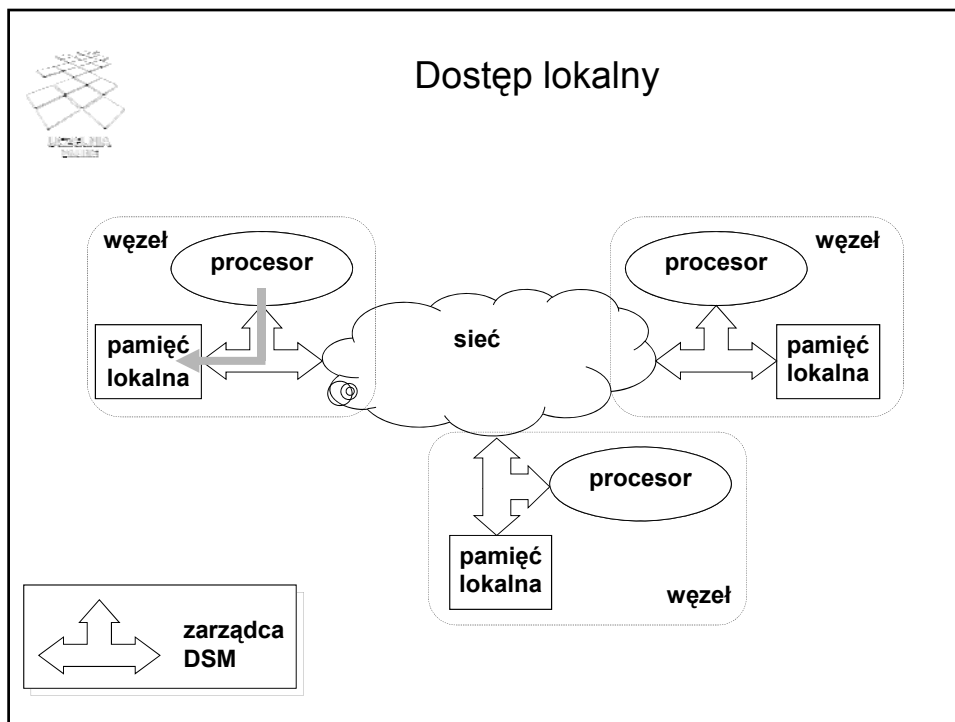
DSM (ang. *Distributed Shared Memory*) — wspólna wirtualna przestrzeń adresowa, dostępna dla wszystkich węzłów systemu rozproszonego.

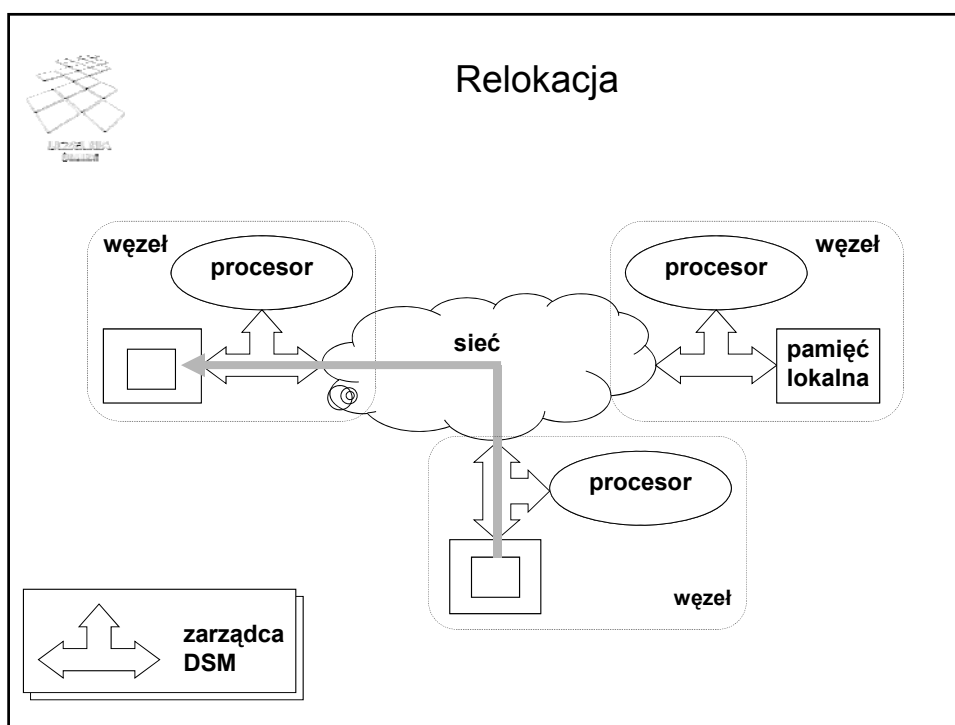
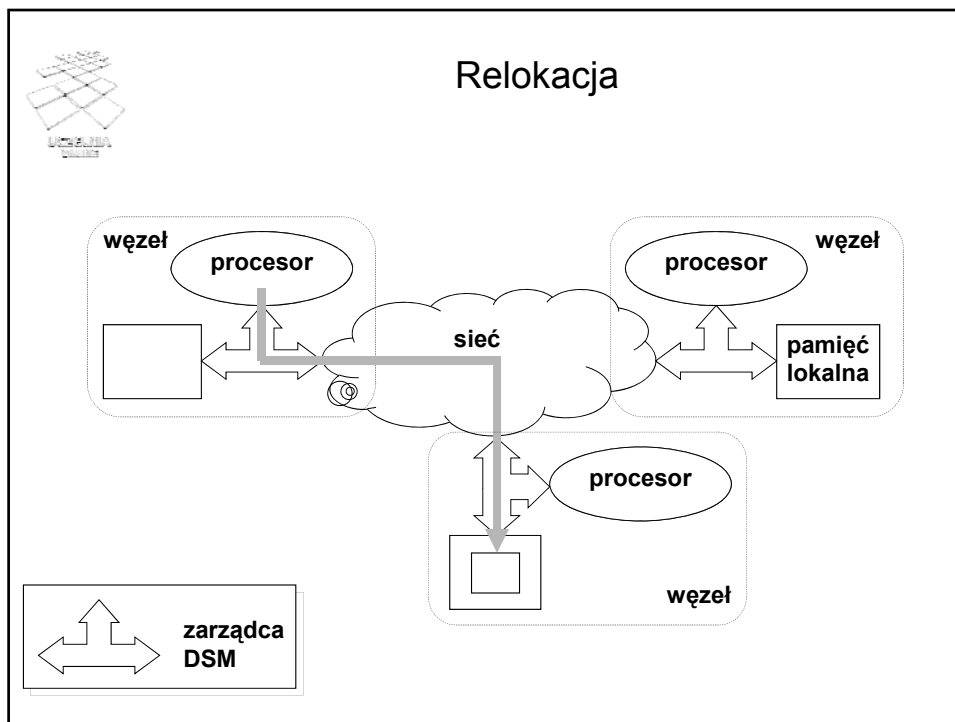
Zalety

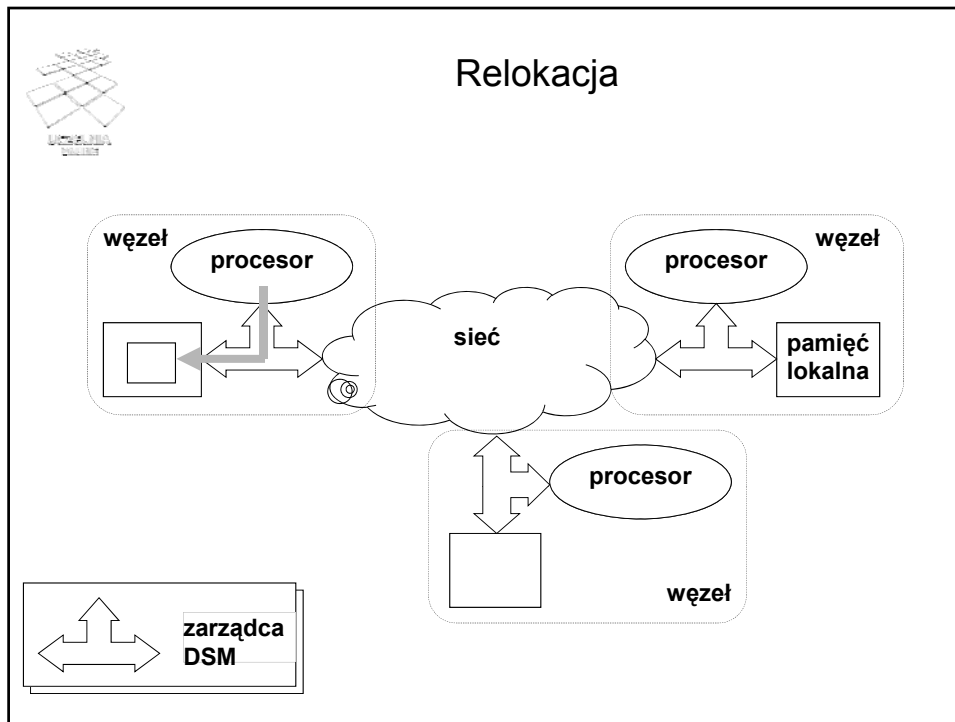
- wygodny paradygmat programowania równoległego
- skalowalność i łatwość rozbudowy
- dostęp do fizycznej pamięci wszystkich węzłów
- środowisko uruchomieniowe dla programów równoległych pisanych dla maszyn wieloprocesorowych







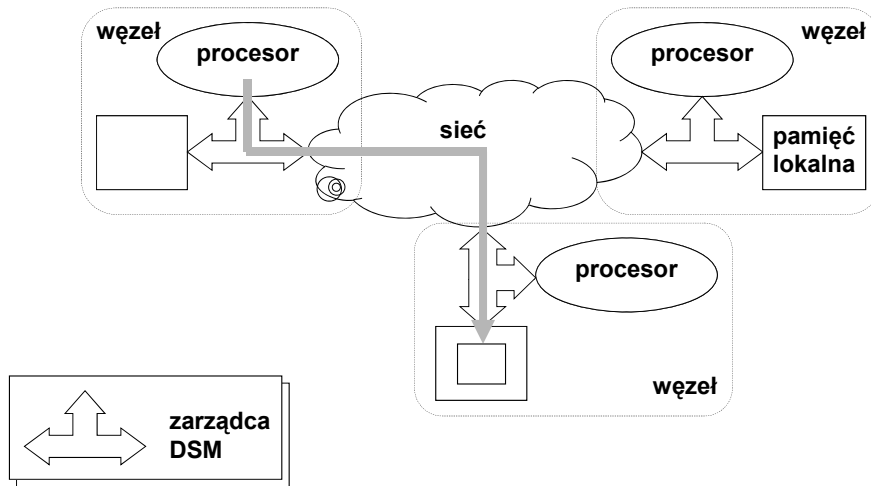




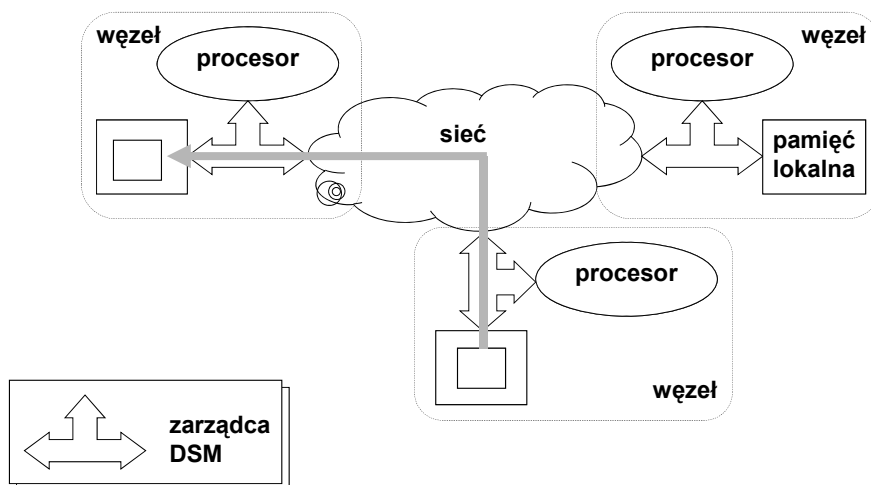
- 
- ### Relokacja — charakterystyka
1. Problem lokalizacji
 - adres obiektu zmienienia się w czasie
 2. Problem rozmiaru i struktury przemieszczanej jednostki
 - małe obiekty → duży poziom współdzielenia
 - duże obiekty → mały narzut administracyjny
 3. Problem migotania (ang. *trashing*, *ping-pong effect*)
 - naprzemienne odwołania kilku procesów

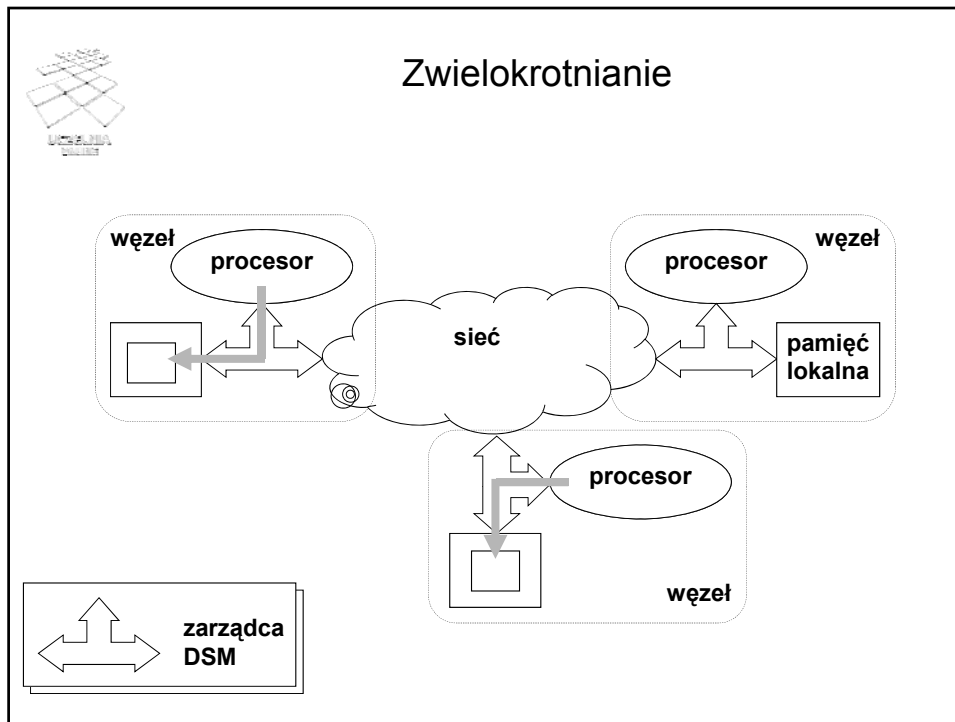


Zwielokrotnianie



Zwielokrotnianie





- 
- ### Zwielokrotnianie — charakterystyka
- Problem lokalizacji
 - tworzenie nowych replik
 - usuwanie starych replik
 - Problem rozmiaru i struktury obiektów zwielokrotnianych
 - Problem migotania nie występuje
 - kopia dla każdego ubiegającego się węzła
 - Problem spójności kopii (replik)
 - stosunek liczby zapisów do odczytów

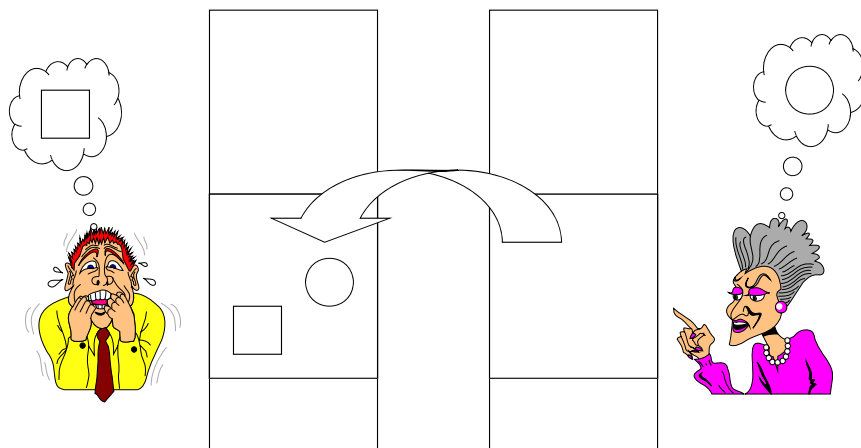


Struktura zwielokrotnianej jednostki

- **Strona** – fizyczne połączenie kilku odrębnych obiektów logicznych w jedną jednostkę udostępnianą jako całość przez **DSM** (*problem fałszywego współdzielenia*)
- **Pojedyncza zmienna** – duży jednostkowy koszt relokacji i utrzymywania spójności
- **Obiekt** (hermetyczna struktura danych udostępniana tylko przez zdefiniowane metody) – możliwość optymalizacji w strategii utrzymywania spójności w związku ze ściśle określonym sposobem dostępu (poprzez metody)



Fałszywe współdzielenie





Protokół koherencji

Protokół koherencji (spójności) jest algorytmem rozproszonym realizującym określony model spójności

1. Protokół **unieważniania** danych (ang. *invalidation protocol*)
 - małe komunikaty
 - jednokrotnie unieważnienie
2. Protokół **aktualizacji** danych (ang. *update protocol*) – niespójne repliki są aktualizowane
 - większe komunikaty



Model spójności

Model spójności określa gwarancje dotyczące spójności replik, dawane aplikacji (równoległej) przez system

- W jaki sposób definiować model spójności?
- W jaki sposób określić gwarancje dla aplikacji?
- Kiedy i w jaki sposób egzekwować te gwarancje?



Spójność ścisła

Spójność ścisła (ang. *strict consistency*)

- każdy odczyt zmiennej x zwraca wartość odpowiadającą wynikowi *ostatnio* wykonanej operacji zapisu

Systemy rozproszone

- niejednoznaczność określenia *ostatni* (brak globalnego zegara)
- duży koszt realizacji spójności ścisłej



Modele spójności replik

Modele spójności nastawione na dane

- modele spójności przy dostępie ogólnym
uspójnianie danych przy każdej modyfikacji
- modele spójności przy dostępie synchronizowanym
uspójnianie danych tylko podczas wykonywania jawnych operacji synchronizujących

Modele spójności nastawione na klienta

- uwzględnienie mobilności klienta



Modele spójności przy dostępie ogólnym

- **Spójność atomowa** (*ang. atomic consistency*)
liniowość (*ang. linearizability*)
- **Spójność sekwencyjna** (*ang. sequential consistency*)
- **Spójność przyczynowa** (*ang. causal consistency*)
- **Spójność PRAM** (*ang. pipelined RAM consistency*)
- **Spójność podręczna** (*ang. cache consistency*)
koherencja (*ang. coherence*)
- **Spójność procesorowa** (*ang. processor consistency*)



Modele spójności przy dostępie synchronizowanym

- **Spójność słaba** (*ang. weak consistency*)
- **Spójność zwalniania** (*ang. release consistency*)
- **Spójność wejścia** (*ang. entry consistency*)
- **Spójność zakresu** (*ang. scope consistency*)



Podstawowe założenia

- W skład systemu DSM wchodzi:
 - zbiór sekwencyjnych procesów $P = \{p_1, p_2, \dots, p_n\}$
 - zbiór współdzielonych zmiennych $X = \{x_1, x_2, \dots\}$
- Każdy proces ma własną replikę całego zbioru X
- Proces p_i może realizować na zmiennej $x \in X$ operacje:
 - zapisu wartości v , oznaczane jako $w_i(x)v$
 - odczytu wartości v , oznaczane jako $r_i(x)v$
- Realizacja operacji przebiega w dwóch fazach:
 - żądanie operacji (ang. *operation issue*)
 - wykonanie operacji (ang. *operation execution*)



Oznaczenia

$w_i(x)v$	zapis wartości v do zmiennej x wykonany przez proces p_i
$r_i(x)v$	odczyt wartości v ze zmiennej x wykonany przez proces p_i
O	zbiór wszystkich operacji w systemie
O_i	zbiór operacji procesu p_i (żądanych przez p_i)
OW	zbiór wszystkich operacji zapisu w systemie
$O x$	zbiór wszystkich operacji na zmiennej x
$\rightarrow_i$	lokalny porządek operacji procesu p_i
$\rightarrow$	porządek przyczynowy
$\mapsto_i$	uszeregowanie operacje postrzegane są przez proces p_i



Definicja porządku przyczynowego

$$(1) \quad \forall_{a1, a2 \in Q} \{c1 \rightarrow a1, c2 \rightarrow a1 \rightarrow c1 \rightarrow a2\}$$

$$(2) \quad \forall_{x \in I} w(x) \rightarrow r(x)$$

$$(3) \quad \forall_{a1, a2 \in \Pi} \{c1 \rightarrow c2 \wedge c1 \rightarrow c2\} \rightarrow a1 \rightarrow a2$$



Definicja uszeregowania legalnego

- Uszeregowanie $\mapsto_i$ jest legalne $\Leftrightarrow$

UWAGA

W celu uproszczenia **definicji** zakłada się, że każda operacja zapisu danej zmiennej zapisuje unikalną wartość, co umożliwia identyfikowanie operacji zapisu poprzez tą wartość



Definicja historii

Historia lokalna (procesu p_i)

Zbiór liniowo uporządkowany $h_i = (O_i, \rightarrow_i)$

Historia globalna

Zbiór częściowo uporządkowany $h = (O, \rightarrow)$

Obraz historii h w procesie p_i

Zbiór liniowo uporządkowany $h_{v_i} = (O_i \cup OW, \mapsto_i)$

Obraz historii h

Kolekcja obrazów procesów: $h_v = \langle h_{v_1}, h_{v_2}, \dots, h_{v_n} \rangle$



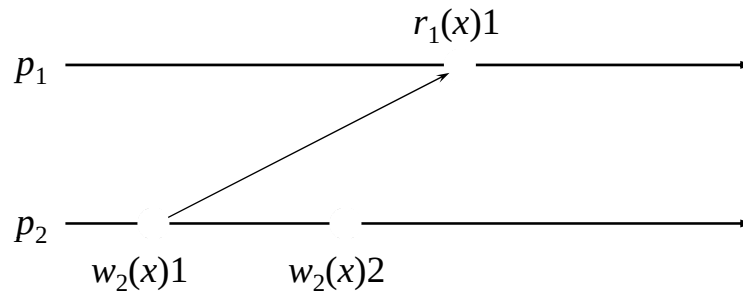
Spójność sekwencyjna

Obraz h_v historii h musi spełniać następujące warunki:



Spójność sekwencyjna – przykład

$hv_1: w_2(x)1 \mapsto_1 r_1(x)1 \mapsto_1 w_2(x)2$



$hv_2: w_2(x)1 \mapsto_2 w_2(x)2$



Algorytm *fast-read*

- Protokół spójności dla modelu sekwencyjnego
- Algorytm dla procesu p_i
- **upon read(x)**
return $M_i[x]$
- **upon receipt of $U(x, v)$ from p_k**
 $M_i[x] := v$
 if $k = i$
 signal
 end if
- **upon write(x, v)**
 atomic_broadcast $U(x, v)$
 wait



Algorytm *fast-write*

- upon read(x)
if $num_i \neq 0$
wait ◦
end if
return $M_i[x]$
- upon receipt of $U(x, v)$ from p_k
 $M_i[x] := v$
if $k = i$
 $num_i := num_i - 1$
 if $num_i = 0$
 ◦ signal
 end if
end if
- upon write(x, v)
 $num_i := num_i + 1$
FIFO_atomic_broadcast $U(x, v)$



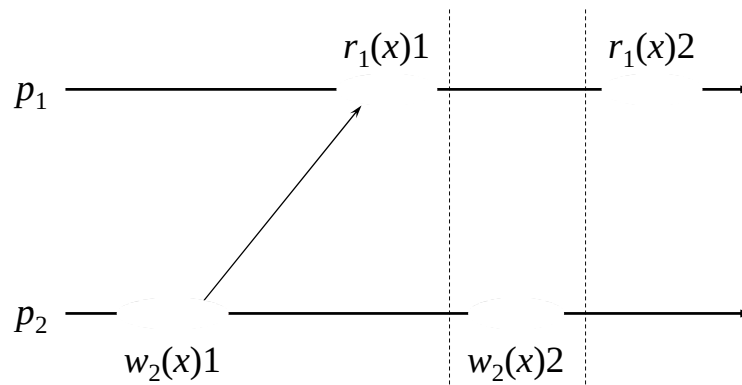
Spójność atomowa

Obraz $h\nu$ historii h musi spełniać następujące warunki:

$o1$ kończy się **w czasie rzeczywistym**,
zanim zaczyna się $o2$



Spójność atomowa — przykład



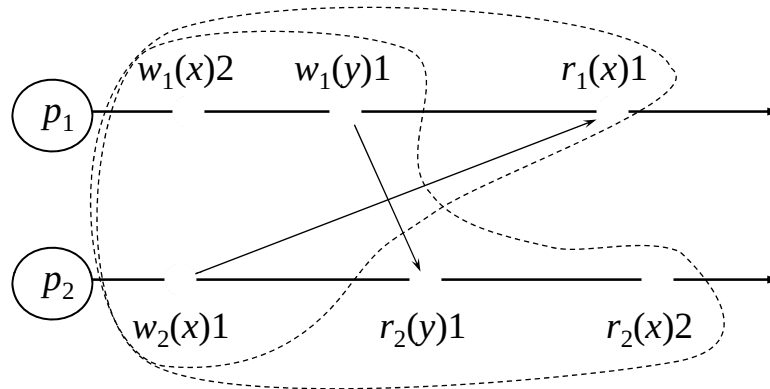
Spójność przyczynowa

Obraz $h\nu$ historii h musi spełniać następujący warunek:



Spójność przyczynowa — przykład

$hv_1: w_1(x)2 \mapsto_1 w_1(y)1 \mapsto_1 w_2(x)1 \mapsto_1 r_1(x)1$



$hv_2: w_2(x)1 \mapsto_2 w_1(x)2 \mapsto_2 w_1(y)1 \mapsto_2 r_2(y)1 \mapsto_2 r_2(x)2$



Protokół dla spójności przyczynowej

Algorytm dla procesu p_i

- **upon read(x)**
return $M_i[x]$
- **write(x, v)**
 $M_i[x] := v$
causal_broadcast $U(x, v)$
- **upon receipt of $U(x, v)$ from p_k**
if $k \neq i$
 $M_i[x] := v$
end if



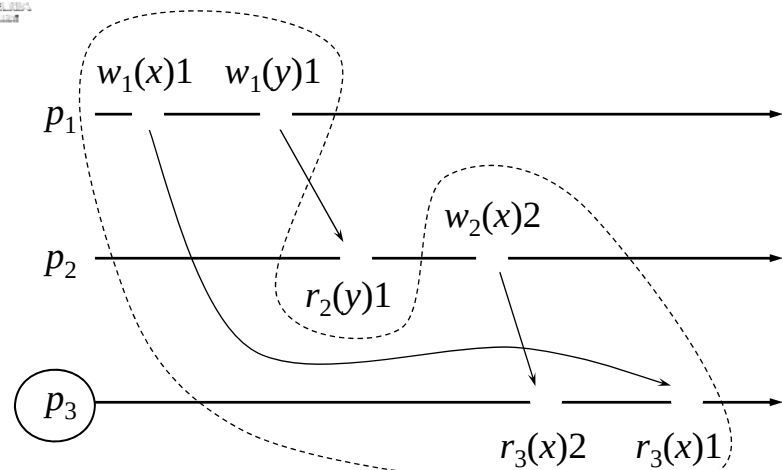
Spójność PRAM

PRAM — pipelined RAM

Obraz $h\nu$ historii h musi spełniać następujący warunek:



Spójność PRAM — przykład



$h\nu_3: \textcircled{w_2(x)2} \mapsto_3 r_3(x)2 \mapsto_3 \textcircled{w_1(x)1} \mapsto_3 r_3(x)1 \mapsto_3 w_1(y)1$



Protokół dla spójności PRAM

Algorytm dla procesu p_i

- **upon read(x)**
return $M_i[x]$
- **upon write(x, v)**
 $M_i[x] := v$
FIFO_broadcast $U(x, v)$
- **upon receipt of $U(x, v)$ from p_k**
if $k \neq i$
 $M_i[x] := v$
end if

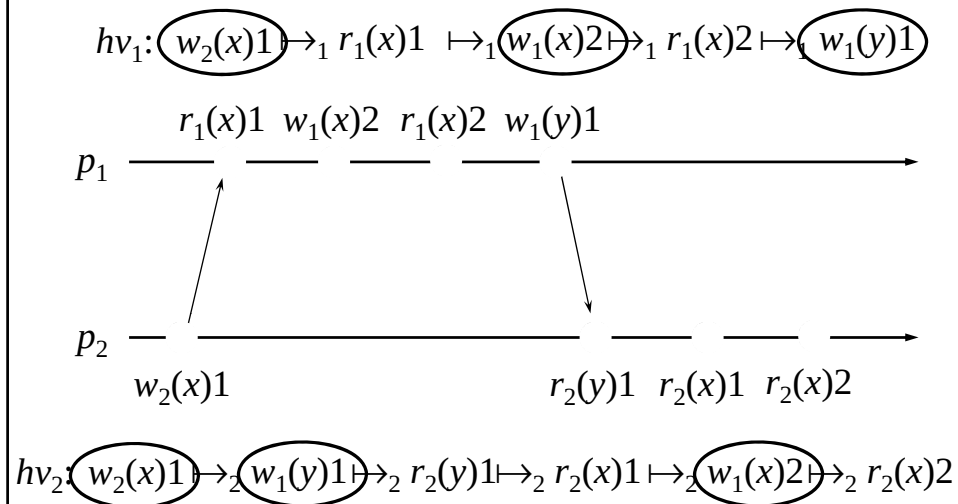


Spójność podręczna

Obraz h_v historii h musi spełniać następujący warunek:



Spójność podręczna — przykład



Protokół dla spójności podręcznej (1)

Algorytm *fast-read* dla procesu p_i :

- **upon read(x)**
return $M_i[x]$
- **upon receipt of $U(x, v)$**
from p_k
 $M_i[x] := v$
if $k = i$
signal
end if
- **upon write(x, v)**
atomicx_broadcast $U(x, v)$
wait



Protokół dla spójności podręcznej (2)

Algorytm *fast-write* dla procesu p_i :

- **upon read(x)**
if $num_i[x] \neq 0$
wait $\circ$ —————
end if
return $M_i[x]$
- **upon write(x, v)**
 $num_i[x] := num_i[x] + 1$
FIFO_x_atomicx_broadcast $U(x, v)$
- **upon on receipt of $U(x, v)$ from p_k**
 $M_i[x] := v$
if $k = i$
 $num_i[x] := num_i[x] - 1$
 if $num_i[x] = 0$
 signal $\circ$
 end if
end if

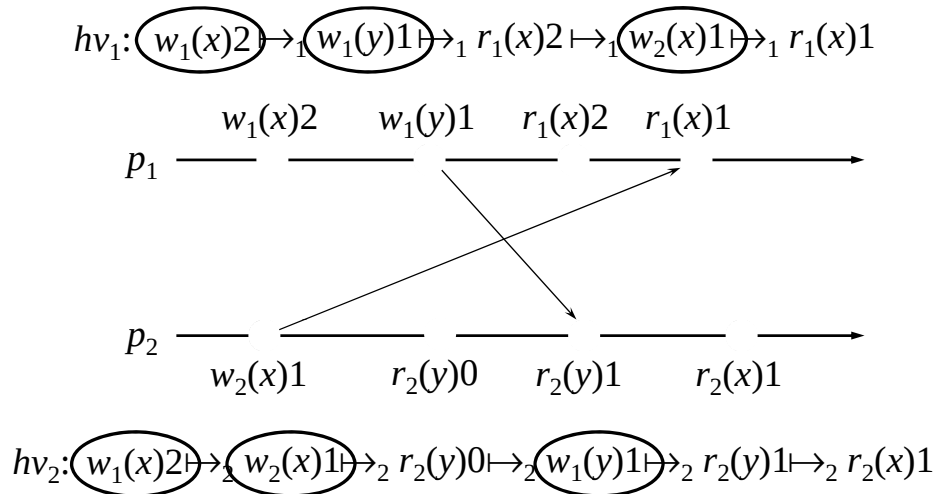


Spójność procesorowa

Obraz h_v historii h musi spełniać następujące warunki
(PRAM + spójność podręczna):



Spójność procesorowa — przykład



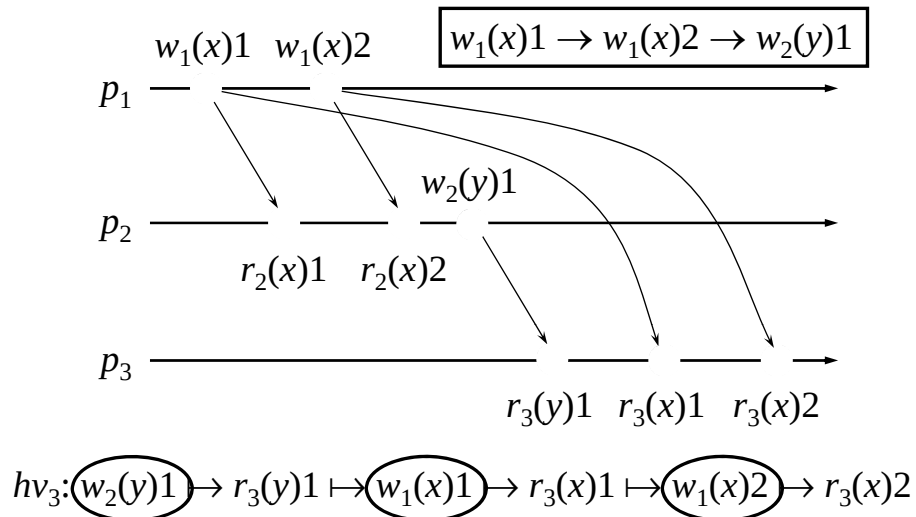
Protokół dla spójności procesorowej

Algorytm *fast-write* dla procesu p_i :

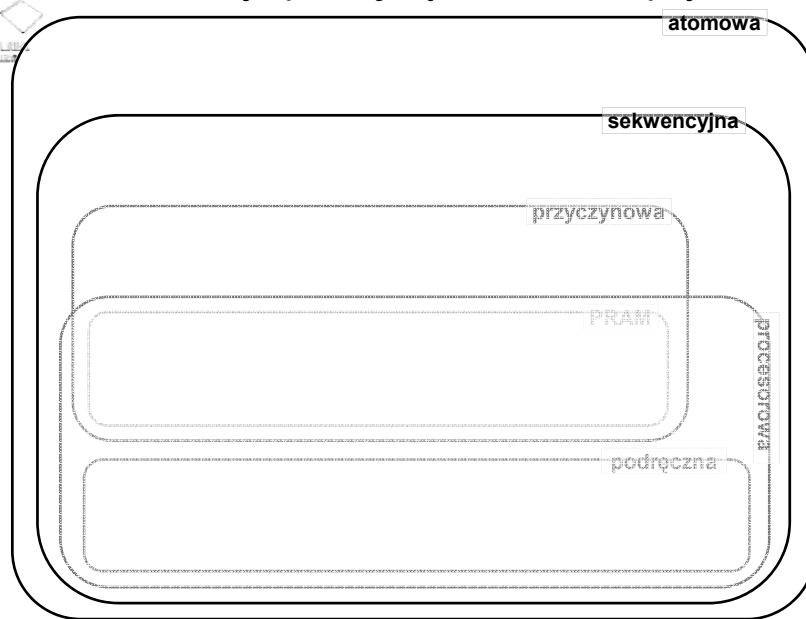
- **upon read(x)**
if $num_i[x] \neq 0$
wait $\bullet$
end if
return $M_i[x]$
- **upon write(x, v)**
 $num_i[x] := num_i[x] + 1$
FIFO_atomicx_broadcast $U(x, v)$
- **upon receipt of $U(x, v)$**
from p_k
 $M_i[x] := v$
if $k = i$
 $num_i[x] := num_i[x] - 1$
if $num_i[x] = 0$
signal $\circ$
end if
end if



Naruszenie porządku przyczynowego



Relacje pomiędzy modelami spójności





Zadanie

W jakich modelach spójności wskazana operacja odczytu $r_3(x)v$

1. zwróci wartość $v = 1$,
2. zwróci wartość $v = 2$,
3. zwróci wartość $v = 3$,

